

Cours
Programmation Orientée
Objet Avancée

Chapitre 2
La généricité

Public Cible : L2DSI

Réalisé par : KHELIFA Afifa

Année universitaire 2018-2019



PLAN

01

Problématique

02

Définition d'une classe générique

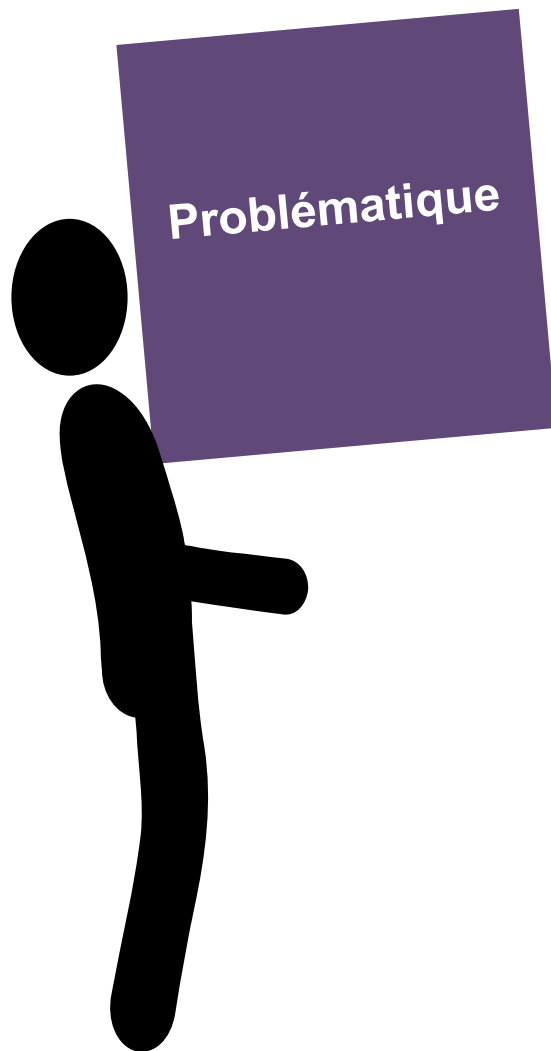
04

Les méthodes génériques

05

Types paramétrés constraints (BOUNDED)

Problématique



Exemple : Paire d'éléments

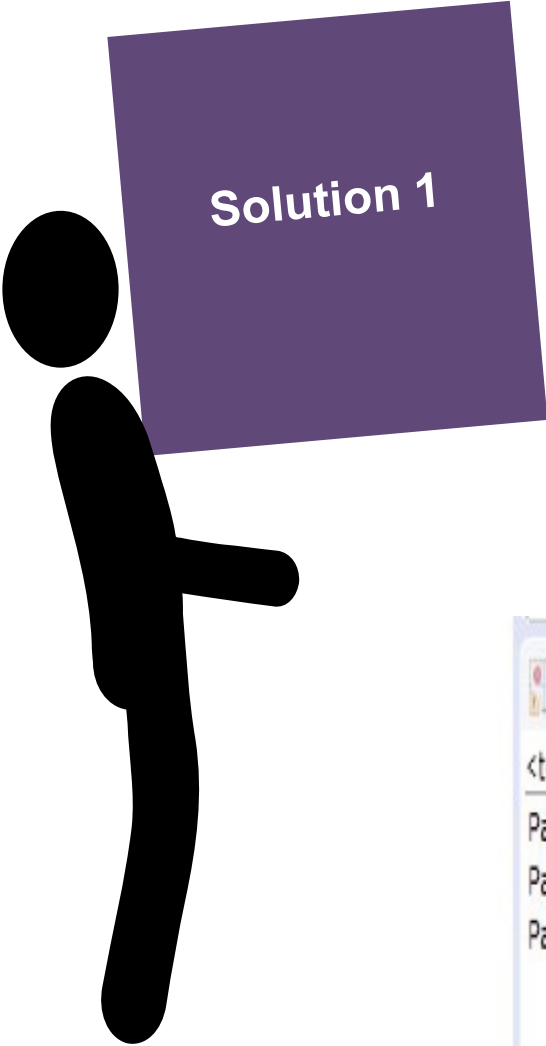
- Soit à gérer des paires d'entiers, de chaînes et d'étudiants.

Solution 1 : écrire 3 classes, chacune correspondante à un type de données :

- **PaireInt** : qui permet de gérer une paire d'entiers.
- **PaireCh** : qui permet de gérer une paire de chaînes.
- **PaireEtud** : qui permet de gérer une paire d'étudiants.

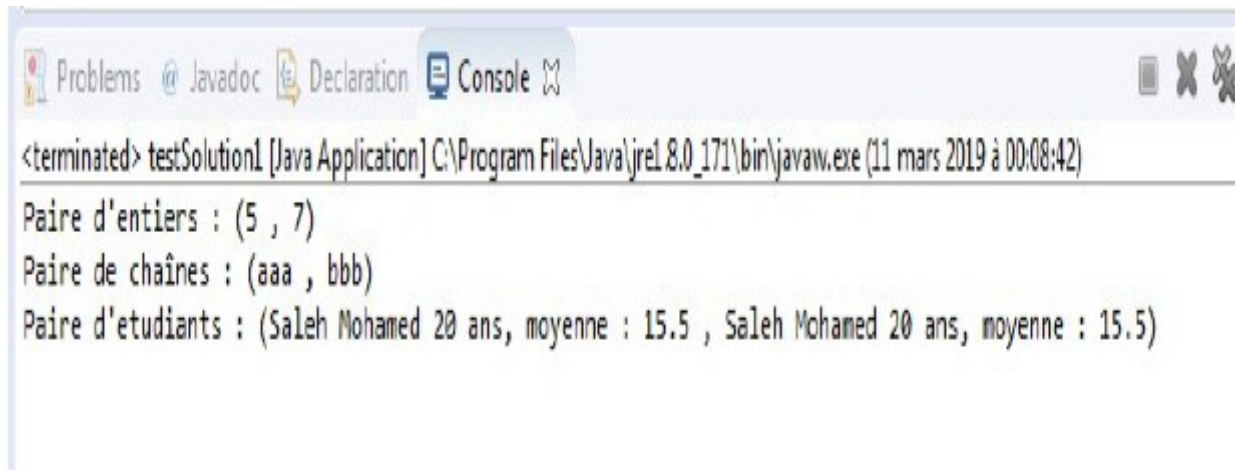
✗ **Inconvénient** : Redondance du code

Problématique



Solution 1

```
package Solution1;
public class testSolution1 {
    public static void main(String[] args) {
        PaireInt p1 = new PaireInt(5,7);
        PaireCh p2= new PaireCh("aaa", "bbb");
        Etudiant e1 = new Etudiant("Karim", "Selimi", 19, 12.3);
        Etudiant e2 = new Etudiant("Mohamed", "Saleh", 20, 15.0);
        PaireEtud p3 = new PaireEtud(e1, e2);
        System.out.println ("Paire d'entiers : " + p1);
        System.out.println ("Paire de chaînes : " + p2);
        System.out.println ("Paire d'etudiants : " + p3);
    }
}
```



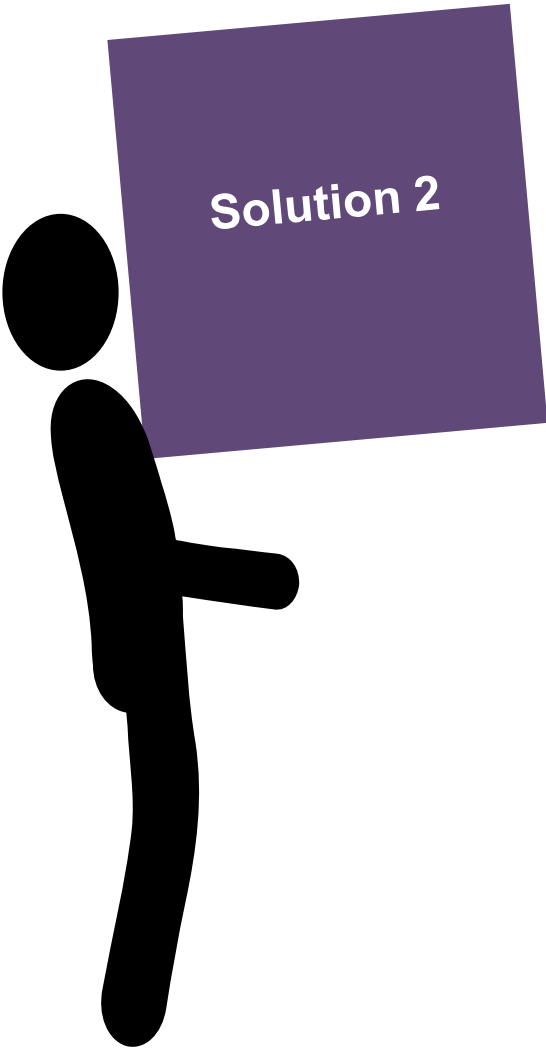
Problems @ Javadoc Declaration Console

<terminated> testSolution1 [Java Application] C:\Program Files\Java\jre1.8.0_171\bin\javaw.exe (11 mars 2019 à 00:08:42)

Paire d'entiers : (5 , 7)
Paire de chaînes : (aaa , bbb)
Paire d'etudiants : (Saleh Mohamed 20 ans, moyenne : 15.5 , Saleh Mohamed 20 ans, moyenne : 15.5)

Problématique

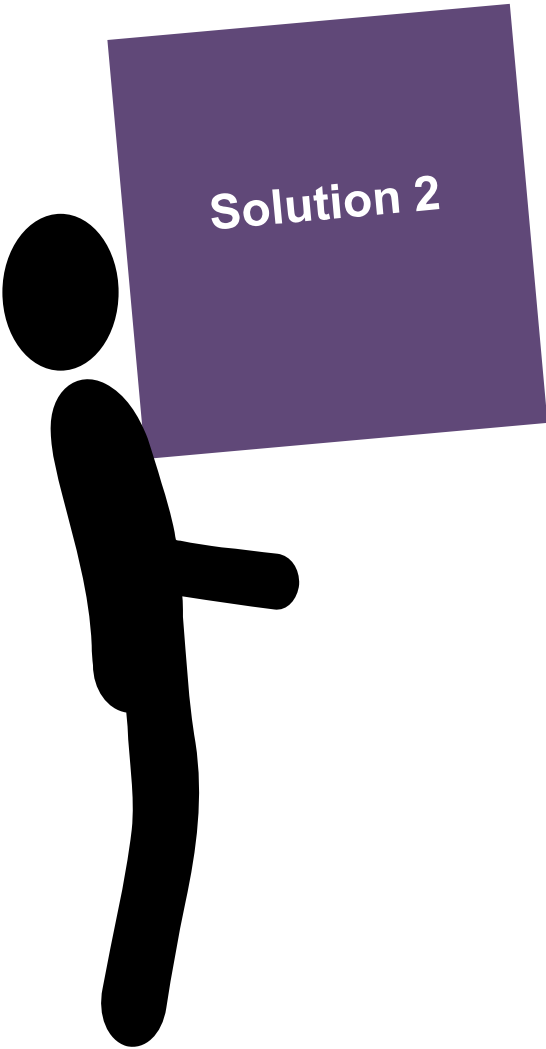
Ecrire une seule classe qui gère le type le plus général (Object dans notre cas)



Solution 2

```
1 package Solution2;
2 public class PaireObj {
3     Object premier;
4     Object second;
5     public PaireObj (Object p, Object s)
6     {
7         premier = p;
8         second=s;
9     }
10    public Object getPremier()
11    {
12        return premier;
13    }
14    public Object getSecond()
15    {
16        return second;
17    }
18    public void setPremier (Object p) {
19        premier = p;
20    }
21    public void setSecond(Object s)
22    {
23        second = s;
24    }
25    public String toString ()
26    {
27        return "(" + premier.toString() + " , " + second.toString() + ")";
28    }
```

Problématique



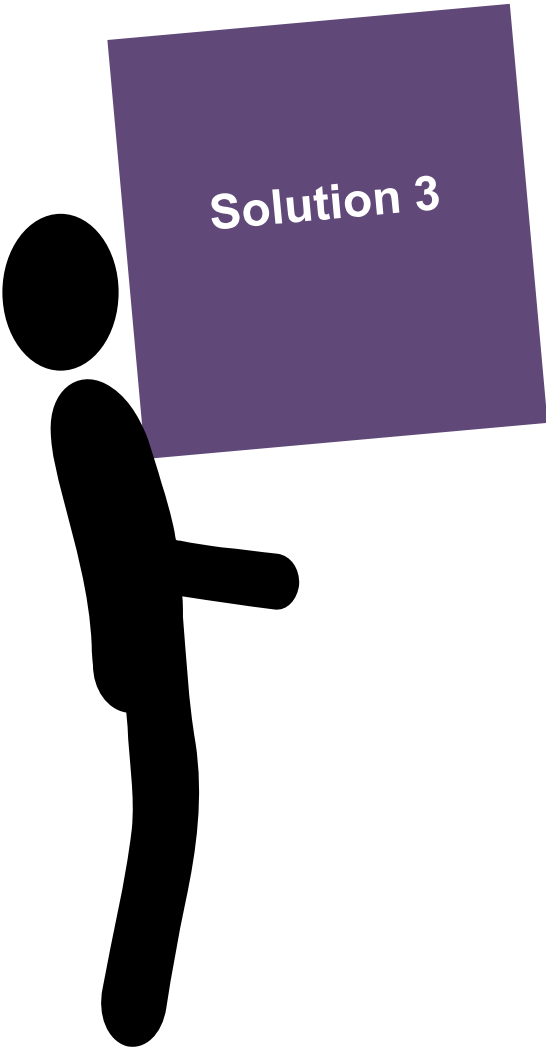
Solution 2

```
1 package Solution2;
2 import Solution1.Etudiant;
3 public class testSolution2 {
4     public static void main(String[] args) {
5         PaireObj p1 = new PaireObj(5,7);
6         PaireObj p2= new PaireObj("aaa", "bbb");
7         Etudiant e1 = new Etudiant("Karim", "Selimi", 19, 12.3);
8         Etudiant e2 = new Etudiant("Mohamed", "Saleh", 20, 15.0);
9         PaireObj p3 = new PaireObj(e1, e2);
10        System.out.println ("Paire d'entiers : " + p1);
11        System.out.println ("Paire de chaînes : " + p2);
12        System.out.println ("Paire d'etudiants : " + p3);
13    }
14 }
```

Limites de la solution 2

```
13 int x = (int)p1.getPremier();
14 String s = p2.getSecond(); //erreur il faut faire un cast
15 int y = (int)p2.getPremier(); //Exception à l'exécution
```

Problématique



Solution 3

Solution 3 : Utiliser la généricité
Nous allons reprendre notre classe **PaireObj**
en la rendant générique :

```
1 package Solution3;  
2 public class Paire <T>{  
3     T premier;  
4     T second;  
5     public Paire (T p, T s)  
6     {  
7         premier = p;  
8         second=s;  
9     }  
10    public T getPremier()  
11    {  
12        return premier;  
13    }  
14    public T getSecond()  
15    {  
16        return second;  
17    }  
18    public void setPremier (T pr)  
19    {  
20        premier = pr;  
21    }  
22    public void setSecond (T s)  
23    {  
24        second = s;  
25    }  
26    public String toString ()  
27    {  
28        return "(" + premier.toString() + " , " + second.toString() + ")";  
}
```


Problématique

8

Solution 3

Solution 3 (Test)

```
1 package Solution3;
2 import Solution1.Etudiant;
3 public class testSolution3 {
4     public static void main(String[] args) {
5         Paire<Integer> p1 = new Paire<Integer>(5,7);
6         Paire<String> p2= new Paire<String>("aaa", "bbb");
7         Etudiant e1 = new Etudiant("Karim", "Selimi", 19, 12.3);
8         Etudiant e2 = new Etudiant("Mohamed", "Saleh", 20, 15.0);
9         Paire<Etudiant> p3 = new Paire<Etudiant>(e1, e2);
10        System.out.println ("Paire d'entiers : " + p1);
11        System.out.println ("Paire de chaînes : " + p2);
12        System.out.println ("Paire d'etudiants : " + p3);
13    }
14 }
```

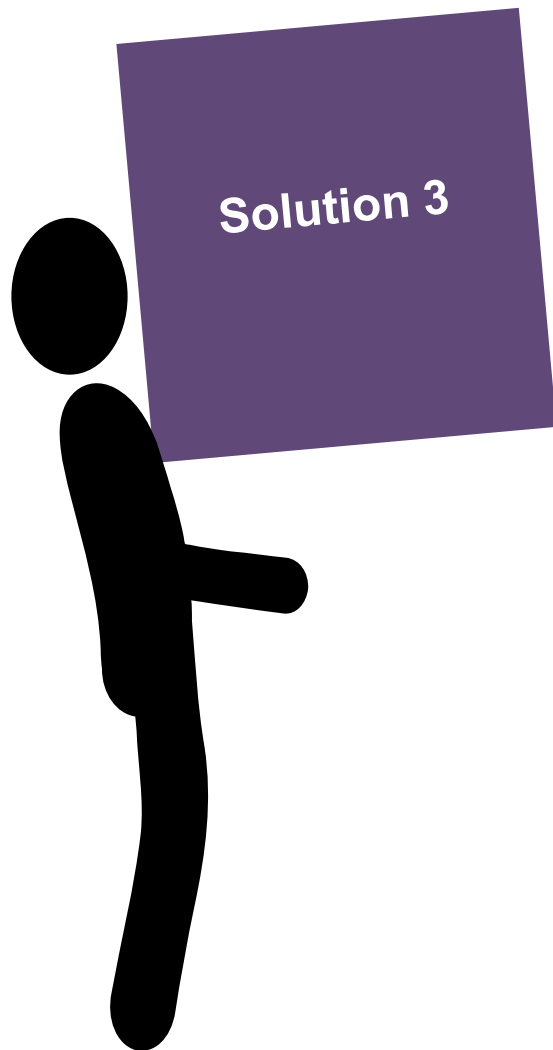
<terminated> testSolution3 [Java Application] C:\Program Files\Java\jre1.8.0_171\bin\javaw.exe (11 mars 2019 à 00:55:16)

Paire d'entiers : (5 , 7)

Paire de chaînes : (aaa , bbb)

Paire d'etudiants : (Selimi Karim 19 ans, moyenne : 12.3 , Saleh Mohamed 20 ans, moyenne : 15.0)

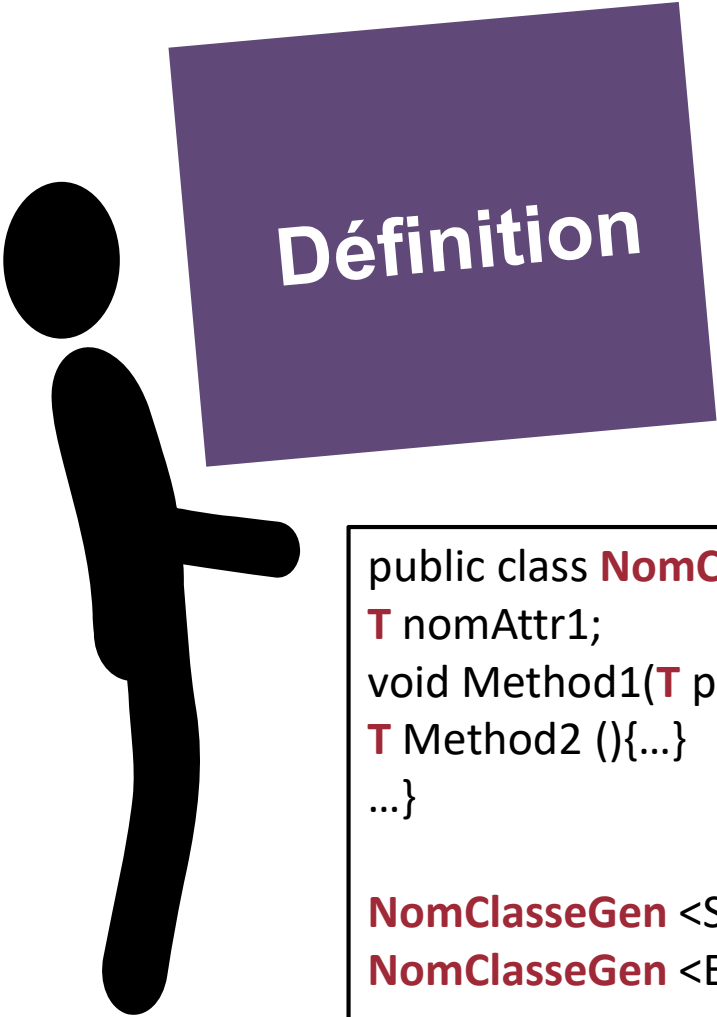
Problématique



La Généricité permet d'écrire un code :

- Plus pratique
- Plus simple
- Plus sûr
- Plus puissant

DÉFINITION D'UNE CLASSE GÉNÉRIQUE SIMPLE 10



Définition

Une classe générique est une classe comprenant une ou plusieurs variables de TYPE : <T>

```
public class NomClasseGen <T> {  
    T nomAttr1;  
    void Method1(T par1){...}  
    T Method2 () {...}  
    ...}
```

```
NomClasseGen <String> obj1;  
NomClasseGen <Etudiant> obj2;
```

- **obj1** est un objet de type **NomClasseGen** <String>.
- Le paramètre de la classe **NomClasseGen** est de type **String**.
- Tout ce qui fait référence à T est remplacé par **String**.

- **obj2** est un objet de type **NomClasseGen** <Etudiant>.
- Le paramètre de la classe **NomClasseGen** est de type **Etudiant**.
- Tout ce qui fait référence à T est remplacé par **Etudiant**.

DÉFINITION D'UNE CLASSE GÉNÉRIQUE SIMPLE

11

Classe
générique à 2
variables de
type

```
1 package Exemples;
2 public class Couple <T, S>{
3     private T premier;
4     private S second;
5     public Couple (T a, S b)
6     {
7         premier = a;
8         second = b;
9     }
10    public T getPremier()
11    {
12        return premier;
13    }
14    public S getSecond()
15    {
16        return second;
17    }
18    public void setPremier(T v)
19    {
20        premier = v;
21    }
22    public void setSecond(S v)
23    {
24        second = v;
25    }
26    public String toString()
27    {
28        return "(" + premier.toString() + ", " + second.toString() + ")";
```

DÉFINITION D'UNE CLASSE GÉNÉRIQUE SIMPLE

12

Classe
générique à 2
variables de
type

Test de la classe Couple

```
1 package Exemples;
2 import Solution1.Etudiant;
3 import Solution3.Paire;
4 public class testCouple {
5     public static void main(String[] args) {
6         Couple <Integer, Integer> c1 = new Couple <Integer, Integer>(5,7);
7         System.out.println ("Couple d'entiers: " + c1);
8         Couple <String, Integer> occ = new Couple <String, Integer>("aa", 3);
9         System.out.println (occ.getPremier() + " se trouve " + occ.getSecond() + " fois");
10        Etudiant e1 = new Etudiant("Karim", "Selimi", 19, 12.3);
11        Etudiant e2 = new Etudiant("Mohamed", "Saleh", 20, 15.0);
12        Paire<Etudiant> p3 = new Paire<Etudiant>(e1, e2);
13        Couple <Paire, String> sout = new Couple<Paire, String>(p3, "10/06/2019");
14        System.out.println (sout);
15    }
16 }
```

Problems @ Javadoc Declaration Console

```
<terminated> testCouple [Java Application] C:\Program Files\Java\jre1.8.0_171\bin\javaw.exe (11 mars 2019 à 01:39:56)
Couple d'entiers: (5, 7)
aa se trouve 3 fois
((Selimi Karim 19 ans, moyenne : 12.3 , Saleh Mohamed 20 ans, moyenne : 15.0), 10/06/2019)
```

Méthodes génériques



Méthodes
génériques

Les **méthodes génériques** peuvent être définies dans des classes ordinaires ou dans des classes génériques.

Quand on déclare une méthode générique, le paramètre de type est déclaré avant le type de retour et après la portée (public, private) et l'indication d'une méthode de classe (static).

Syntaxe

<Visibilité> <static> <**Variabletype**> TypeRetour nomMethode (....)

Exemples

```
public static <T> T selectionner (T v1, T v2) {...}
```

```
public static <T> void Affiche (T[] tab) {...}
```

Méthodes génériques

14

eclipse-workspace - testExemplesGenericité/src/Methodes/UtilTableau.java - Eclipse IDE

File Edit Source Refactor Navigate Search Project Run Window Help

```
2 import java.lang.Math;
4 public class UtilTableau {
5     public static <T> void Swap (T[] tab, int i, int j)
6     {
7         T aux;
8         aux = tab[i];
9         tab[i] = tab[j];
10        tab[j] = aux;
11    }
12
13    public static <T> T AuHasard (T[] tab)
14    {
15        Double r = Math.random();
16        int pos = (int) (r* tab.length);
17        return tab[pos];
18    }
19
20    public static <T> void Affiche (T[] tab, int nb)
21    {
22        int i;
23        System.out.println();
24        for (i=0; i<nb; i++)
25        {
26            System.out.print (tab[i] + " ");
27        }
28    }
29
30    public static void main (String[] argv)
31    {
32        Integer t1 [] = {5,7,1,12,20};
33        String t2[] = {"aa", "bb", "a", "zz", "hh"};
34        UtilTableau.Affiche(t1, t1.length);
35        UtilTableau.Affiche(t2, t2.length);
36        System.out.println ("\nAu hasard, nous avons la valeur " + UtilTableau.AuHasard(t1));
37        System.out.println ("\nAu hasard, nous avons la valeur " + UtilTableau.AuHasard(t2));
38        System.out.println ("\nAprès permutation de la case 0 et 2, le tableau est :");
39        UtilTableau.Swap(t1, 2, 0);
40        UtilTableau.Affiche(t1, t1.length);
41    }
42 }
```

<terminated> UtilTableau [Java Application] C:\Program Files\Java\jre-9\bin

```
[
5 7 1 12 20
aa bb a zz hh
Au hasard, nous avons la valeur 12

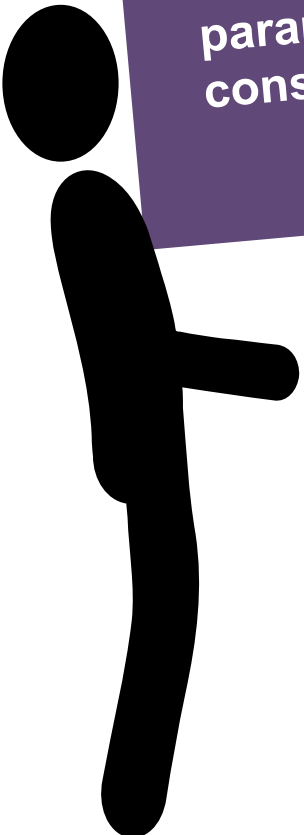
Au hasard, nous avons la valeur aa

Après permutation de la case 0 et 2, le tableau est :

1 7 5 12 20
```

Activer Windows
Accédez aux paramètres

Types paramétrés constraints



Types
paramétrés
constraints

Pourquoi des types constraints ?

- Lorsqu'on utilise un paramètre de type formel T, on ne peut rien supposer sur le type T (c'est un type quelconque).
- Si une variable ou un paramètre **v** est déclaré de type T, aucune méthode ne peut être appelée sur v (à part celles qui sont dans la classe Object).

Syntaxe

- Soit T un paramètre de type d'une classe, d'une interface ou d'une méthode générique On peut indiquer que T doit hériter d'une classe mère M (au sens large ; T pourra être de type la classe Mere) en utilisant la syntaxe suivante:

<T extends Mere>

- Ou qu'il doit implémenter (ou qu'il doit hériter d'une ou plusieurs interfaces :

< T extends I1 & I2>

Types paramétrés constraints



Types
paramétrés
constraints

Exemples

```
<T extends Number>  
<T extends List<String>>  
<T extends ArrayList<? extends Number>>  
<T extends E>  
<A extends Personne, B extends A>
```